



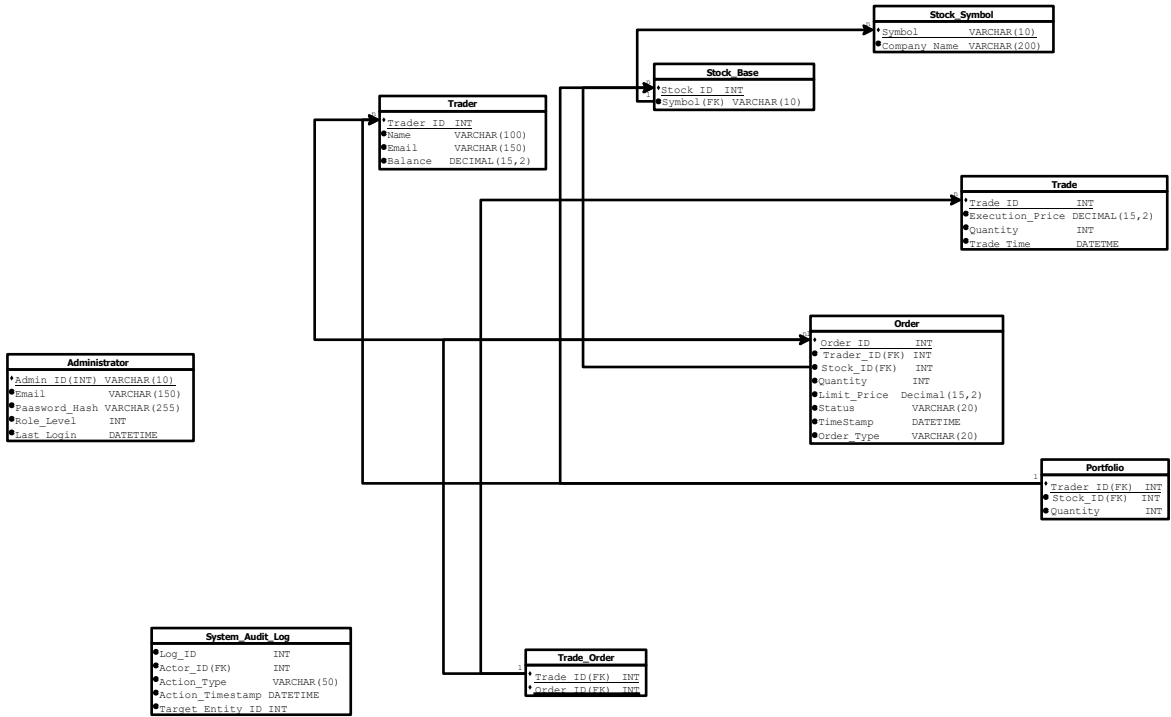
Dhirubhai Ambani
University
Technology

Formerly DA-IICT

MINI STOCK MANAGEMENT SYSTEM

NORMALIZATION REPORT

Janvi Patel - 202401447



Mini Stock Management System: Database Architecture & Normalization Report

Executive Summary

This report details the relational database design for a Mini Stock Management System. The architecture models the core realities of a trading environment, capturing the interactions between Traders, Stocks, Orders, and the resulting execution Trades. To guarantee data integrity and prevent update anomalies, the schema has been rigorously analyzed for Functional Dependencies and normalized up to Boyce-Codd Normal Form (BCNF).

Section 1: Base Relational Schema

The initial database structure was derived directly from the system's Entity-Relationship constraints.

1.1 Entity Classification

The system relies on strong entities to represent independent domain concepts, and bridge tables to resolve complex many-to-many (M:N) trading mechanics.

- **Strong Entities:** Trader, Stock, Administrator, Trade, Order, System_Audit_Log.
- **M:N Bridge Tables:** * Portfolio (Resolves the relationship of Traders maintaining multiple Stocks).
 - Trade_Order (Resolves the reality that every Trade execution involves matching two separate Orders).

1.2 Initial Schema Definition

Below is the baseline schema notation prior to normalization. Primary keys are underlined, and foreign keys are explicitly marked.

- **Trader** (Trader_ID, Name, Email, Balance)
- **Stock** (Stock_ID, Symbol, Company_Name)
- **Administrator** (Admin_ID, Email, Password_Hash, Role_Level, Last_Login)
- **Order** (Order_ID, *Trader_ID*, *Stock_ID*, Quantity, Limit_Price, Status, Timestamp, Order_Type)
- **Trade** (Trade_ID, Execution_Price, Quantity, Trade_Time)
- **Portfolio** (*Trader_ID*, *Stock_ID*, Quantity)
- **Trade_Order** (*Trade_ID*, *Order_ID*)
- **System_Audit_Log** (Log_ID, Actor_ID, Action_Type, Action_Timestamp, Target_Entity_ID)

Section 2: Functional Dependencies (FDs)

To ensure the database behaves predictably, we mapped the semantic rules of the trading floor into formal Functional Dependencies. An FD dictates that knowing the value of one attribute (or set of attributes) strictly determines the value of another.

Core Non-Trivial Dependencies

- **Trader:** Trader_ID uniquely determines the Trader's Name, Email, and Balance. Email acts as an alternate candidate key.
 - **Stock:** Stock_ID determines both Symbol and Company_Name. Additionally, a specific Symbol determines the Company_Name.
 - **Order:** Order_ID represents a distinct market action, determining the exact Trader_ID, Stock_ID, Quantity, Limit_Price, Status, Timestamp, and Order_Type.
 - **Trade:** Trade_ID represents an executed match, determining the final Execution_Price, matched Quantity, and Trade_Time.
 - **Portfolio:** The composite key (Trader_ID, Stock_ID) is required to determine the Quantity of shares held.
 - **System_Audit_Log:** The surrogate Log_ID tracks distinct system events, determining the Actor_ID responsible, the Action_Type, the Action_Timestamp, and the affected Target_Entity_ID.
-

Section 3: Canonical Cover Optimization

Before normalizing, we optimized the functional dependencies to remove redundant routing that would waste database calculation time.

During this process, we identified a transitive dependency in the Stock entity. The dependency Stock_ID → Company_Name is redundant because it can be derived transitively (Stock_ID → Symbol, and Symbol → Company_Name). We stripped this redundancy to form a clean Minimal Cover, ensuring no extraneous attributes remained.

The Minimal FD Set:

Trader: Trader_ID → Name | Email | Balance; Email → Trader_ID Stock_ID → Symbol; Stock_ID → Company_Name

Administrator: Admin_ID → Email | Password_Hash | Role_Level | Last_Login; Email → Admin_ID

Order: Order_ID → Trader_ID | Stock_ID | Quantity | Limit_Price | Status | Timestamp | Order_Type

Trade: Trade_ID → Execution_Price | Quantity | Trade_Time

Portfolio: (Trader_ID, Stock_ID) → Quantity Admin_ID, Log_ID → | Action_Type | Action_Timestamp | Target_Entity_ID

Trade_Order: (No non-trivial FDs)

System_Audit_Log: Log_ID → Actor_ID | Action_Type | Action_Timestamp | Target_Entity_ID

Section 4: Boyce-Codd Normal Form (BCNF) Analysis

A robust financial database must be resilient against insertion, deletion, and update anomalies. We evaluated every relation against the BCNF requirement: *For every non-trivial functional dependency, the determinant must be a superkey.*

4.1 BCNF Validation

- The **Trader**, **Administrator**, **Order**, **Trade**, **Portfolio**, **Trade_Order**, and **System_Audit_Log** relations all natively satisfy BCNF. In each case, their non-trivial functional dependencies rely strictly on their respective primary or candidate keys.
- **The Stock Anomaly:** The original Stock relation failed BCNF. The dependency Symbol -> Company_Name exists, but Symbol is not a superkey of the full Stock table. If left uncorrected, updating a company's name would require modifying multiple rows, risking data corruption.

4.2 Structural Decomposition

To enforce BCNF compliance, we decomposed the flawed Stock table into two highly cohesive sub-relations:

1. **Stock_Base** (Stock_ID, Symbol)
2. **Stock_Symbol** (Symbol, Company_Name)

Proof of Integrity:

- **Lossless Join:** Because the shared attribute (Symbol) serves as the primary key for the new Stock_Symbol relation, joining these tables perfectly reconstructs the original dataset without generating phantom records (Heath's Theorem).
- **Dependency Preservation:** All original functional dependencies are fully enforced within the boundaries of these two new tables.

Section 5: Final Production Schema

The optimized, BCNF-compliant schema consists of 9 distinct relations.

Relation	Primary Key	Foreign Keys	Key Attributes & Types
Trader	Trader_ID	None	Name (VARCHAR), Email (VARCHAR), Balance (DECIMAL)
Stock_Base	Stock_ID	Symbol -> Stock_Symbol	Symbol (VARCHAR)
Stock_Symbol	Symbol	None	Company_Name (VARCHAR)
Administrator	Admin_ID	None	Email, Password_Hash, Role_Level, Last_Login (DATETIME)
Order	Order_ID	Trader_ID -> Trader	Quantity (INT), Limit_Price (DECIMAL), Status, Order_Type, Timestamp

Relation	Primary Key	Foreign Keys	Key Attributes & Types
		Stock_ID -> Stock_Base	
Trade	Trade_ID	None	Execution_Price (DECIMAL), Quantity (INT), Trade_Time
Portfolio	(Trader_ID, Stock_ID)	Trader_ID -> Trader Stock_ID -> Stock_Base	Quantity (INT)
Trade_Order	(Trade_ID, Order_ID)	Trade_ID -> Trade Order_ID -> Order	Pure Bridge Table
System_Audit_Log	Log_ID	None (Polymorphic Actor)	Actor_ID (INT), Action_Type, Action_Timestamp, Target_Entity_ID

Section 6: SQL Data Definition Language (DDL)

The following script executes the physical creation of the normalized architecture, complete with domain constraint checks, cascading updates, and restricted deletions to protect referential integrity.

```
CREATE DATABASE mini_stock_management;
```

```
USE mini_stock_management;
```

```
CREATE TABLE Trader (
```

```
Trader_ID INT NOT NULL,  
Name VARCHAR(100) NOT NULL,  
Email VARCHAR(150) NOT NULL,  
Balance DECIMAL(15, 2) NOT NULL DEFAULT 0.00,  
CONSTRAINT PK_Trader PRIMARY KEY (Trader_ID),  
CONSTRAINT UQ_Trader_Email UNIQUE (Email)  
);
```

```
CREATE TABLE Stock_Symbol (  
    Symbol VARCHAR(10) NOT NULL,  
    Company_Name VARCHAR(200) NOT NULL,  
    CONSTRAINT PK_Stock_Symbol PRIMARY KEY (Symbol)  
);
```

```
CREATE TABLE Stock_Base (  
    Stock_ID INT NOT NULL,  
    Symbol VARCHAR(10) NOT NULL,  
    CONSTRAINT PK_Stock_Base PRIMARY KEY (Stock_ID),  
    CONSTRAINT UQ_Stock_Symbol UNIQUE (Symbol),  
    CONSTRAINT FK_StockBase_Symbol FOREIGN KEY (Symbol) REFERENCES Stock_Symbol(Symbol)  
        ON DELETE RESTRICT ON UPDATE CASCADE  
);
```

```
CREATE TABLE Administrator (  
    Admin_ID INT NOT NULL,  
    Email VARCHAR(150) NOT NULL,  
    Password_Hash VARCHAR(255) NOT NULL,  
    Role_Level INT NOT NULL DEFAULT 1,  
    Last_Login DATETIME NULL,  
    CONSTRAINT PK_Administrator PRIMARY KEY (Admin_ID),  
    CONSTRAINT UQ_Admin_Email UNIQUE (Email)  
);
```

```
CREATE TABLE Trade (  
    Trade_ID INT NOT NULL,  
    Execution_Price DECIMAL(15, 2) NOT NULL,
```

```
Quantity INT NOT NULL,  
Trade_Time DATETIME NOT NULL,  
CONSTRAINT PK_Trade PRIMARY KEY (Trade_ID)  
);
```

```
CREATE TABLE `Order` (  
    Order_ID INT NOT NULL,  
    Trader_ID INT NOT NULL,  
    Stock_ID INT NOT NULL,  
  
    Quantity INT NOT NULL,  
    Limit_Price DECIMAL(15, 2) NOT NULL,  
    Status VARCHAR(20) NOT NULL  
        CHECK (Status IN ('PENDING', 'OPEN', 'FILLED', 'CANCELLED', 'PARTIAL')),  
    Timestamp DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    Order_Type VARCHAR(20) NOT NULL  
        CHECK (Order_Type IN ('BUY', 'SELL', 'LIMIT', 'MARKET', 'STOP')),  
    CONSTRAINT PK_Order PRIMARY KEY (Order_ID),  
    CONSTRAINT FK_Order_Trader FOREIGN KEY (Trader_ID) REFERENCES Trader(Trader_ID)  
        ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT FK_Order_Stock FOREIGN KEY (Stock_ID) REFERENCES Stock_Base(Stock_ID)  
        ON DELETE RESTRICT ON UPDATE CASCADE  
);
```

```
CREATE TABLE Portfolio (  
    Trader_ID INT NOT NULL,  
    Stock_ID INT NOT NULL,  
    Quantity INT NOT NULL DEFAULT 0,  
    CONSTRAINT PK_Portfolio PRIMARY KEY (Trader_ID, Stock_ID),  
    CONSTRAINT FK_Portfolio_Trader FOREIGN KEY (Trader_ID) REFERENCES Trader(Trader_ID)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    CONSTRAINT FK_Portfolio_Stock FOREIGN KEY (Stock_ID) REFERENCES Stock_Base(Stock_ID)  
        ON DELETE CASCADE ON UPDATE CASCADE  
);
```



```
CREATE TABLE Trade_Order (  
    Trade_ID INT NOT NULL,  
    Order_ID INT NOT NULL,  
    CONSTRAINT PK_Trade_Order PRIMARY KEY (Trade_ID, Order_ID),  
    CONSTRAINT FK_TradeOrder_Trade FOREIGN KEY (Trade_ID) REFERENCES Trade(Trade_ID)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    CONSTRAINT FK_TradeOrder_Order FOREIGN KEY (Order_ID) REFERENCES `Order`(Order_ID)  
        ON DELETE CASCADE ON UPDATE CASCADE  
);
```

```
CREATE TABLE System_Audit_Log (  
    Log_ID INT NOT NULL AUTO_INCREMENT,  
    Actor_ID INT NOT NULL,  
    Action_Type VARCHAR(50) NOT NULL,  
    Action_Timestamp DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    Target_Entity_ID INT NULL,  
    CONSTRAINT PK_System_Audit_Log PRIMARY KEY (Log_ID)  
);
```